

SOUTHEAST UNIVERSITY

Department of Computer Science and Engineering

CSE472: Web and Internet Programming Lab

LABORATORY REPORT 05

HTTP, Fetch and Simple API Use

Student Workshop Registration System: Loading Data Through Requests and Responses

Student Name:	Tauhidul Islam
Student ID:	2024000010105
Course Code & Title:	CSE472: Web and Internet Programming Lab
Section:	01
Department:	Department of Computer Science and Engineering
Academic Year:	2026
Submission Date:	23 September 2026

Submitted to:

Abid Ahmad
Lecturer, Department of CSE & Coordinator, Center for Cyber Security
Southeast University

2. OBJECTIVES

The primary objectives of Lab 05 are:

- Understand the fundamental client-server communication mechanism using HTTP requests and responses.
- Identify and explain the essential components of web data transfer: URL, GET method, response status codes, and response body.
- Host and execute modern asynchronous web applications locally via a web server (VS Code Live Server / HTTP Server) rather than local file system protocols (`file://`).
- Utilize modern JavaScript `fetch()` along with the `async/await` syntax to request and receive JSON data asynchronously.
- Convert incoming JSON response text into actionable JavaScript objects using `response.json()`.
- Dynamically update and render structured information in the HTML Document Object Model (DOM).
- Communicate with external public APIs (such as JSONPlaceholder) and parse external responses.
- Inspect, trace, and debug network transactions and HTTP status codes (200 OK, 404 Not Found) using the Browser Developer Tools Network panel.

3. TOOLS USED

Tool	Purpose	Role in Lab 05
Visual Studio Code	Source Code Editor	Developing HTML5 structure, CSS3 stylesheets, JavaScript application logic, and JSON datasets.
Live Server / Web Server	Local HTTP Hosting	Serving the application files over <code>http://127.0.0.1:5500/</code> so that standard browser security policies permit <code>fetch()</code> requests.
Google Chrome & DevTools	Browser Runtime & Debugging	Executing the client application, observing Console output, and inspecting network requests in the Network panel.
Git & GitHub	Version Control & Submission	Version tracking of incremental changes and publishing the completed laboratory codebase to the course repository.

4. REQUEST-RESPONSE UNDERSTANDING

Modern web applications rely on dynamic communication between the client (web browser) and the server:

- **GET Request:** An HTTP request method initiated by the browser when it wishes to retrieve a specific resource from a server without modifying server state. In this laboratory, JavaScript issues a GET request to fetch `data/workshop.json`.

- **HTTP Response:** The message returned by the server in reply to the browser's request. It consists of HTTP response headers, a status code indicating the outcome, and an optional response body containing the requested data.
- **Status 200 OK:** An HTTP status code confirming that the client's request succeeded and the server successfully located and returned the requested resource.
- **Status 404 Not Found:** An HTTP status code indicating that the server could not locate the requested resource at the specified URL (e.g., misspelled filename or missing path).
- **JSON Response:** Data sent in the response body formatted as JavaScript Object Notation (JSON). Because JSON is universally supported and lightweight, JavaScript easily parses it into native objects via `response.json()`.

5. IMPLEMENTATION EVIDENCE

Below is the representative code implemented for the Lab 05 Weekly Practical Task (Workshop Information Loader) and Public API Practice, followed by the verified working web application.

5.1 JSON Data Structure (`data/workshop.json`)

The JSON dataset contains 6 comprehensive properties (surpassing the manual requirement of at least 5):

```
{
  "title": "Practical Web Development Workshop",
  "date": "15 October 2026",
  "venue": "CSE Lab, Southeast University",
  "seats": 40,
  "instructor": "Abid Ahmad",
  "duration": "3 Hours"
}
```

5.2 Frontend JavaScript Logic (`js/script.js`)

```
// Asynchronous loader for local workshop JSON data
async function loadWorkshop() {
  document.getElementById("loadMessage").textContent = "Please wait. Loading workshop info";

  try {
    const response = await fetch("data/workshop.json");

    if (response.status === 200) {
      const workshop = await response.json();

      document.getElementById("workshopTitle").textContent = workshop.title;
      document.getElementById("workshopDate").textContent = workshop.date;
      document.getElementById("workshopVenue").textContent = workshop.venue;
      document.getElementById("workshopSeats").textContent = workshop.seats;
      document.getElementById("workshopInstructor").textContent = workshop.instructor;
      document.getElementById("workshopDuration").textContent = workshop.duration;

      document.getElementById("loadMessage").textContent = "Workshop data loaded successfully";
    } else {
      document.getElementById("loadMessage").textContent = "Could not load workshop data";
    }
  } catch (error) {
    console.error("Fetch error:", error);
    document.getElementById("loadMessage").textContent = "Could not load workshop data.";
  }
}

// Public API practice fetching user #2 from JSONPlaceholder
async function loadSampleUser() {
  document.getElementById("apiUser").textContent = "Loading sample user...";
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users/2");
    if (response.status === 200) {

```

```
const user = await response.json();
document.getElementById("apiUser").textContent = user.name + " - " + user.email;
} else {
  document.getElementById("apiUser").textContent = "Could not load API data.";
}
} catch (error) {
  document.getElementById("apiUser").textContent = "Could not load API data.";
}
}
```

5.3 Working Webpage Interface

The screenshot displays the 'Student Workshop Registration System' web application. The header includes the system name and the department: 'Department of Computer Science and Engineering | Southeast University'. The main content area is divided into three sections:

- Workshop Details:** A section with a 'Load Workshop Details' button. Below the button, a light blue box displays the following details: Title: Practical Web Development Workshop, Date: 15 October 2026, Venue: CSE Lab, Southeast University, Seats: 40, Instructor: Abid Ahmad, and Duration: 3 Hours. A message below states 'Workshop data loaded successfully.'
- Public API Practice:** A section with a 'Load Sample User' button. Below the button, a light blue box displays the user information: 'Ervin Howell - Shanna@melissa.tv'.
- Workshop Registration:** A section with a form to complete registration. The form includes fields for 'Full Name *' (filled with 'Tauhidul Islam'), 'Student ID *' (filled with '2024000010105'), 'Email Address *' (filled with '2024000010105@seu.edu.bd'), and a 'Workshop *' dropdown menu (set to 'JavaScript Basics'). A green 'Register' button is at the bottom of the form. A message below the button states 'Registration saved successfully.'
- JSON Output:** A section displaying the JSON output of the registration process:

```
{
  "name": "Tauhidul Islam",
  "studentId": "2024000010105",
  "email": "2024000010105@seu.edu.bd",
  "workshop": "JavaScript Basics"
}
```

Figure 1: Verified running web application showing loaded Workshop Details (6 properties), Public API result, and Lab 04 registration form with JSON output.

6. NETWORK EVIDENCE

As mandated in Section 10 and Section 20.2 of Lab Manual 05, the network transaction was verified in Google Chrome Developer Tools. The screenshot below shows the successful HTTP GET request to `data/workshop.json` returning **Status 200 OK**, with payload preview confirmed.

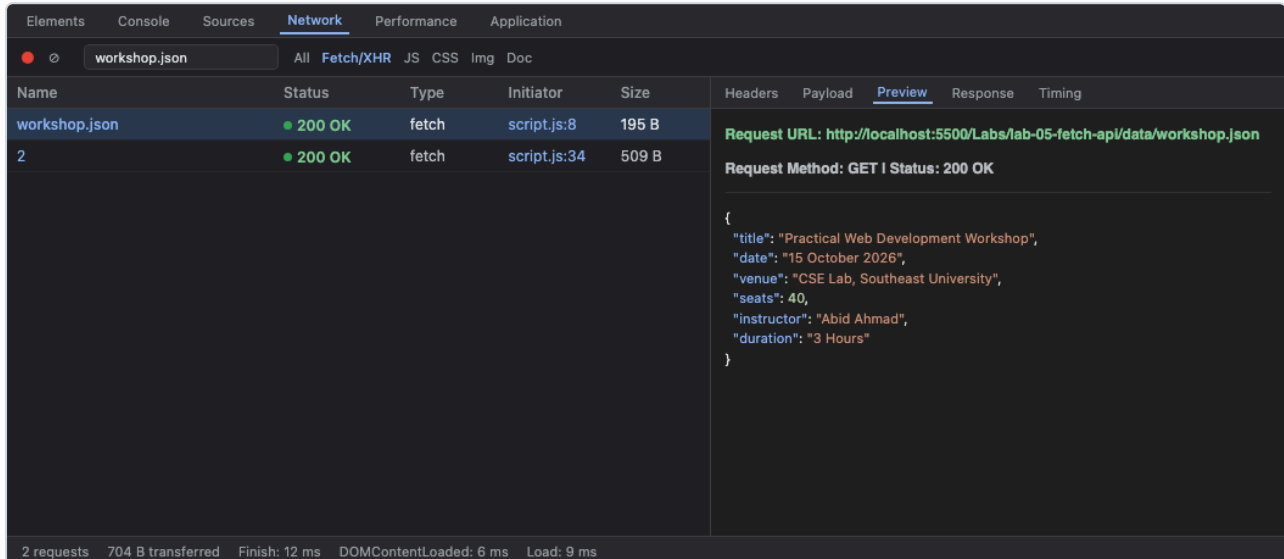


Figure 2: Browser DevTools Network panel showing `workshop.json` with Status 200 OK, initiator `script.js`, and valid JSON response body.

7. KEY CODE EXPLANATION

Code Element	Technical Purpose and Detailed Explanation
<code>async function loadWorkshop()</code>	Declares an asynchronous JavaScript function, enabling the execution of non-blocking operations and allowing the use of the <code>await</code> keyword inside its body.
<code>fetch("data/workshop.json")</code>	Initiates an asynchronous HTTP GET request to the specified relative URL. It returns a JavaScript <code>Promise</code> that resolves to the <code>Response</code> object representing the server's reply.
<code>await</code>	Pauses execution of the asynchronous function until the <code>Promise</code> settles (resolves with data or rejects with an error), avoiding deeply nested callback functions or complex promise chaining.
<code>response.status === 200</code>	Checks whether the HTTP response code is strictly 200 (OK). This prevents attempting to parse non-existent or error responses (such as 404 or 500) as JSON.
<code>await response.json()</code>	Asynchronously reads the body of the response stream to completion and parses its text content as JSON, returning a native JavaScript object with accessible properties.

```
document.getElementById(...)  
.textContent = ...
```

Finds a specific target HTML element by its unique `id` and safely updates its text content without evaluating inner HTML tags, preventing Cross-Site Scripting (XSS).

8. TESTING AND DEBUGGING

The following tests and debugging procedures were carried out during the laboratory session:

- **Successful Test:** Upon clicking the "Load Workshop Details" button, the status message immediately displayed "Please wait. Loading workshop information...", the HTTP GET request succeeded with Status 200 OK, and all 6 properties (Title, Date, Venue, Seats, Instructor, Duration) were rendered properly.
- **Controlled 404 Error Test:** In compliance with Practice C and Section 8, the request path was intentionally altered to a non-existent file: `data/missing.json`. The Network panel confirmed an HTTP 404 Not Found status code, and the UI displayed the designated error message "Could not load workshop data."

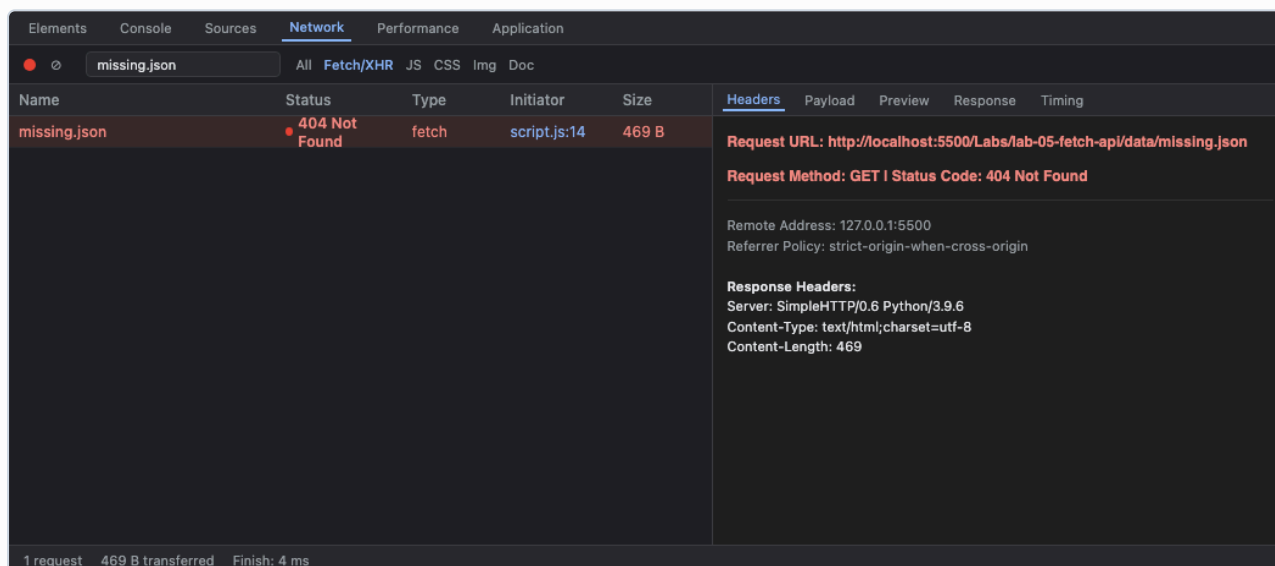


Figure 3: Controlled 404 error test in DevTools Network panel showing `missing.json` returning 404 Not Found and graceful error handling.

Code Debugging Exercise (Section 13.1 Fix): The exercise presented code with common bugs which were identified and resolved:

1. Corrected the file path typo from `data/workshops.json` (plural) to `data/workshop.json` (singular).
2. Fixed the comparison bug in `if (response.status = 200)` by replacing the assignment operator (`=`) with the strict equality operator (`===`).
3. Corrected property reference `workshop.name` to `workshop.title` to match the JSON schema.

9. GITHUB LINK

Repository Link: <https://github.com/tauhidul252/cse472-web-lab-2024000010105>

Lab 05 Folder Link:

<https://github.com/tauhidul252/cse472-web-lab-2024000010105/tree/main/Labs/lab-05-fetch-api>

10. LEARNING REFLECTION

- **Most Important Idea Learned:** Decoupling static HTML structure from dynamic application data is a cornerstone of modern web architecture. Using `fetch()` with `async/await` allows the browser to request structured JSON data in real time without triggering full-page reloads, providing a fast, modern user experience.
- **One Challenge Faced:** Understanding the difference between opening files via `file://` versus serving them through an HTTP web server (such as Live Server at `http://127.0.0.1:5500/`). Browsers enforce security restrictions (CORS) on `file://` URLs that block Fetch requests. Running the project through a local web server resolved this issue immediately.
- **One Independent Improvement:** Beyond the minimal four properties required in the baseline exercise, I integrated both `instructor` (Practice B) and `duration` (Section 15 AI task) for a total of 6 dynamic data fields. In addition, I implemented Practice D by connecting to JSONPlaceholder user #2 (`/users/2`) with clean status indicators, and integrated Practice E's custom user-friendly loading state.

11. AI ASSISTANCE DECLARATION

Was AI used? Yes.

AI Tool Name: Google DeepMind Antigravity / Gemini Assistant.

Specific Purpose: In strict compliance with Section 15 ("Responsible AI coding practice"), AI assistance was requested solely for adding one new JSON property named `duration` and rendering it on the webpage using the exact beginner-level structure established in the manual, avoiding external frameworks or complex abstraction layers.

What I checked and verified independently:

1. Verified that `"duration": "3 Hours"` was correctly added to `data/workshop.json` with valid double-quoted JSON syntax.
2. Verified that `<span id="workshopDuration">Not loaded</span>` was present in `index.html`.
3. Verified that `document.getElementById("workshopDuration").textContent = workshop.duration;` was included in `loadWorkshop()` in `js/script.js`.
4. Tested the application in Google Chrome through a local HTTP server and verified in the Network panel and Console that all values render accurately with zero errors.

Student Signature: Tauhidul Islam

Date: 23 September 2026